

Motivations for No SQL Databases

Here are five reasons why organizations may choose to use a NoSQL database:

1. Scalability: NoSQL databases are designed to scale horizontally, meaning that they can handle large amounts of data and user traffic by adding more commodity hardware. This makes it easier to handle the increasing demands of modern applications, without having to make significant changes to the underlying infrastructure.

2. Performance: NoSQL databases are optimized for handling large volumes of data, which means that they can deliver faster performance compared to traditional relational databases. This is particularly important for applications that require real-time data access and low latency.

3. Flexibility: NoSQL databases are schema-less, meaning that they can handle unstructured and semi-structured data in addition to structured data. This flexibility makes it easier to store and manage a wide variety of data types, including documents, graphs, and key-value pairs.

4. Cost-effectiveness: Because NoSQL databases are designed to run on commodity hardware, they are often more cost-effective than traditional relational databases, which can require expensive hardware and software licenses.

5. Availability: NoSQL databases are designed to handle high levels of traffic and data throughput, which means that they can provide high availability and fault tolerance. This is particularly important for mission-critical applications that require constant uptime and minimal downtime.

Types of NoSQL databases

A database is a collection of structured data or information which is stored in a computer system and can be accessed easily. A database is usually managed by a Database Management System (DBMS).

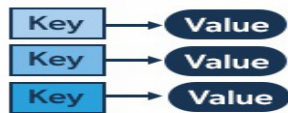
NoSQL is a non-relational database that is used to store the data in the nontabular form. It is a method to database design that focuses on providing a mechanism for storage and retrieval of data that is suited in means other than the tabular relations used in relational databases. NoSQL stands for Not only SQL .

Types of NoSQL Database

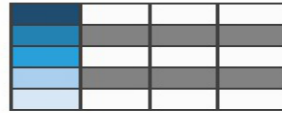
1. Document-based databases
2. Key-value stores
3. Column-oriented databases
4. Graph-based databases

NoSQL

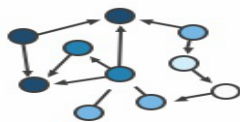
Key-Value



Column-Family



Graph



Document



1. Document-Based Database

The document-based database is a nonrelational database. Instead of storing the data in rows and columns (tables), it uses the documents to store the data in the database. A document database stores data in JSON, BSON, or XML documents.

Documents can be stored and retrieved in a form that is much closer to the data objects used in applications which means less translation is required to use these data in the applications. In the Document database, the particular elements can be accessed by using the index value that is assigned for faster querying.

Collections are the group of documents that store documents that have similar contents. Not all the documents are in any collection as they require a similar schema because document databases have a flexible schema.

- **Key features of documents database:**

- Flexible schema: Documents in the database has a flexible schema. It means the documents in the database need not be the same schema.
- Faster creation and maintenance: the creation of documents is easy and minimal maintenance is required once we create the document.
- No foreign keys: There is no dynamic relationship between two documents so documents can be independent of one another. So, there is no requirement for a foreign key in a document database.
- Open formats: To build a document we use XML, JSON, and others.

2. Key-Value Stores

A key-value store is a nonrelational database. The simplest form of a NoSQL database is a key-value store. Every data element in the database is stored in key-value pairs. The data can be retrieved by using a unique key allotted to each element in the database. The values can be simple data types like strings and numbers or complex objects.

A key-value store is like a relational database with only two columns which is the key and the value.

- **Key features of the key-value store:**

- Simplicity.
- Scalability.
- Speed.

3. Column Oriented Databases

A column-oriented database is a non-relational database that stores the data in columns instead of rows. That means when we want to run analytics on a small number of columns, you can read those columns directly without consuming memory with the unwanted data.

Columnar databases are designed to read data more efficiently and retrieve the data with greater speed. A columnar database is used to store a large amount of data.

- **Key features of Columnar Oriented Database**

- Scalability
- Compression
- Very responsive

4. Graph-Based Databases

Graph-based databases focus on the relationship between the elements. It stores the data in the form of nodes in the database. The connections between the nodes are called links or relationships.

- **Key features of Graph Database**

- In a graph-based database, it is easy to identify the relationship between the data by using the links.
- The Query's output is real-time results.
- The speed depends upon the number of relationships among the database elements.

- Updating data is also easy, as adding a new node or edge to a graph database is a straightforward task that does not require significant schema changes.

Operations in NoSQL

NoSQL is a broad term that describes various database technologies that don't conform to the traditional relational database model. CRUD stands for create, read, update, and delete and refers to the basic functions you'll need to perform on your data. CRUD operations are usually performed using a language like SQL, but NoSQL databases often use different query languages or APIs. This can make it challenging to switch from a traditional relational database to a NoSQL database.

There are many different NoSQL databases, but some of the most popular ones include MongoDB, Cassandra, and Couchbase. And each of these databases have different query languages.

What are CRUD Operations?

Organizations have to keep track of customer data, accounts, payment information, and other records that require persistent storage. The basic operations performed on such data stored in databases are known as CRUD operations. CRUD stands for Create, Read, Update, and Delete.

A relational database has tables with columns and rows. Each row describes a single data point, such as an employee's name, address, phone number, salary, and so on. Each column represents a category of information, such as the employee's age, gender, marital status, and so forth. These data can be manipulated using the CRUD (Create, Read, Update, and Delete) operations in a database. These operations allow users to create records, read existing records, update existing records, and delete existing records.

The operations are as follows:

Create

Read

Update

Delete

1) Create

The Create operations enables users to insert new records into the database. It is same as the functioning of the INSERT function in SQL relational database application. Only an admin can add new attributes to the table. Whereas, a user can create rows and populate them with data cor 1) Creating Documents

The Create in the NoSQL CRUD Operations in MongoDB indicates the creation of a data document.

You can create a document in MongoDB with information about historical monuments, their name, geographical location, country, and city. You can follow the below code snippet for creating the document.

```
{
  "name": "The Pyramids of Giza",
  "city": "Giza",
  "country": "Egypt",
  "gps": {
    "lat": 29.976480,
    "lng": 31.131302
  }
}
```

}responding to each attribute.

2) Read

Similar to the search function, Read enables users to look up particular records in the table and access their values. Users can look for keywords or filter the data based on customized criteria to find records.

Now, you can query the database to retrieve the Document from the database and read its data.

There are two methods to read data in MongoDB

```
find()
findOne()
```

You can use the following command to retrieve all the documents from a database:

```
db.monuments.find()
```

You can also give an objectId as an identifier to return only a specific document:

```
db.monuments.find({"country": "France"}).pretty()
```

3) Update

The Update operation modifies existing records in the database. It is useful when an existing record in the database must be changed along with all the attribute values. You can update one or more existing records into the new record by using the updateOne() or updateMany() functions.

The Update in NoSQL CRUD Operations can be implemented in MongoDB by updating the Documents using the following two methods:

updateOne()

updateMany()

You can use it to update one or all Document(s) in the database. Use the following code to update Arc de Triomphe to Arc de Triomphe de l'Étoile, it's full name.

```
db.monuments.updateOne(
  { "name": "Arc de Triomphe" },
  {
    $set: { "name": "Arc de Triomphe de l'Étoile" }
  }
)
```

4) Delete

Users can delete values with the Delete operation to remove records that are no longer needed. Users can delete one or more records at a time by specifying the right functions i.e, deleteOne() or deleteMany().

The Delete in the NoSQL CRUD Operations can be implemented by deleting records from a MongoDB collection. The method to do this are as follows:

deleteOne()

deleteMany()

You can begin by removing the Arc de Triomphe de l'Étoile monument Document you modified previously:

```
db.monuments.deleteOne(
  { "name": "Arc de Triomphe de l'Étoile" }
```

)

Q. What are NoSQL Databases?

NoSQL is a term used to describe a non-relational database. A NoSQL database does not use the traditional table-based relational database management system (RDBMS). It's often used in Big Data applications because they can scale more easily and handle large amounts of data. They are often more scalable than relational databases. These databases can also offer improved performance as they are designed to be distributed; they can take advantage of parallel processing to speed up data access.

NoSQL databases can also be more flexible because they don't have to adhere to the rigid table structure of an RDBMS. However, NoSQL databases can be more difficult to query and less reliable than relational databases. NoSQL databases are increasing in popularity as more and more organizations look to adopt them for their data storage needs. There are many benefits that NoSQL databases can offer, including scalability, improved performance, and easier management.

Big Data:

Data science is the study of data analysis by advanced technology (Machine Learning, Artificial Intelligence, Big data). It processes a huge amount of structured, semi-structured, and unstructured data to extract insight meaning, from which one pattern can be designed that will be useful to take a decision for grabbing the new business opportunity, the betterment of product/service, and ultimately business growth. Data science process to make sense of Big data/huge amount of data that is used in business. The workflow of Data science is as below:

Objective and the issue of business determining – What is the organization's objective, what level the organization wants to achieve, and what issue the company is facing -these are the factors under consideration. Based on such factors which type of data are relevant is considered.

Collection of relevant data- relevant data are collected from various sources.

Cleaning and filtering collected data – non-relevant data are removed.

Explore the filtered, cleaned data – Finding any hidden pattern, synchronization in data, plotting them in the graph, chart, etc. form that is understandable to a non-technical person.

Creating a model by analyzing data – creating a model, validate it.

Visualization of finding by interpreting data or creating a model for a business person.

Help businesspeople in making the decision and taking the step for the sack of business growth.

Data Mining:

It is a process of extracting insight meaning, hidden patterns from collected data that is useful to take a business decision for the purpose of decreasing expenditure and increasing revenue. **Big Data:** This is a term related to extracting meaningful data by analyzing the huge amount of complex, variously formatted data generated at high speed, that cannot be handled, or processed by the traditional system. **Data Expansion Day by Day:** Day by day amount of data increasing exponentially because of today's various data production sources like a smart electronic devices.

Source of Big Data:

Social Media: Today's world a good percent of the total world population is engaged with social media like Facebook, WhatsApp, Twitter, YouTube, Instagram, etc. Each activity on such media like uploading a photo, or video, sending a message, making comment, putting like, etc create data.

A sensor placed in various places: Sensor placed in various places of the city that gathers data on temperature, humidity, etc. A camera placed beside the road gather information about traffic condition and creates data. Security cameras placed in sensitive areas like airports, railway stations, and shopping malls create a lot of data.

Customer Satisfaction Feedback: Customer feedback on the product or service of the various company on their website creates data. For Example, retail commercial sites like Amazon, Walmart, Flipkart, and Myntra gather customer feedback on the quality of their product and delivery time. Telecom companies, and other service provider organizations seek customer experience with their service. These create a lot of data.

IoT Appliance: Electronic devices that are connected to the internet create data for their smart functionality, examples are a smart TV, smart washing machine, smart coffee machine, smart AC, etc. It is machine-generated data that are created by sensors kept in various devices. For Example, a Smart printing machine – is connected to the internet. A number of such printing machines connected to a network can transfer data within each other. So, if anyone loads a file copy in one printing machine, the system stores that file content, and another printing machine kept in another building or another floor can print out that file hard copy. Such data transfer between various printing machines generates data.

E-commerce: In e-commerce transactions, business transactions, banking, and the stock market, lots of records stored are considered one of the sources of big data. Payments through credit cards, debit cards, or other electronic ways, all are kept recorded as data.

Global Positioning System (GPS): GPS in the vehicle helps in monitoring the movement of the vehicle to shorten the path to a destination to cut fuel, and time consumption. This system creates huge data on vehicle position and movement.

Transactional Data: Transactional data, as the name implies, is information obtained through online and offline transactions at various points of sale. The data contains important information about transactions, such as the date and time of the transaction, the location where it took place, the items bought, their prices, the methods of payment, the discounts or coupons that were applied, and other pertinent quantitative data. These are some of the sources of transactional data: orders for payment, Invoices, E-receipts and recordkeeping etc.

Machine Data: Automatically generated machine data is produced in reaction to an event or according to a set timetable. This indicates that all of the data was compiled from a variety of sources, including satellites, desktop computers, mobile phones, industrial machines, smart sensors, SIEM logs, medical and wearable devices, road cameras, IoT devices, and more. Businesses can monitor consumer behaviour thanks to these sources. Data derived from automated sources expands exponentially in tandem with the market's shifting external environment. These sensors are used to capture this kind of information: In a broader sense, machine data includes data that is generated by servers, user applications, websites, cloud programmes, and other sources.

Handling large datasets using Map-Reduce and Hadoop:

Hadoop:

The definition of a powerful person has changed in this world. A powerful is one who has access to the data. This is because data is increasing at a tremendous rate. Suppose we are living in 100% data world. Then 90% of the data is produced in the last 2 to 4 years. This is because now when a child is born, before her mother, she first faces the flash of the camera. All these pictures and videos are nothing but data. Similarly, there is data of emails, various smartphone applications, statistical data, etc. All this data has the enormous power to affect various incidents and trends. This data is not only used by companies to affect their consumers but also by politicians to affect elections. This huge data is referred to as Big Data. In such a world, where data is being produced at such an exponential rate, it needs to be maintained, analyzed, and tackled. This is where Hadoop creeps in.

Hadoop is a framework of the open source set of tools distributed under Apache License. It is used to manage data, store data, and process data for various big data applications running under clustered systems. In the previous years, Big Data was defined by the “3Vs” but now there are “5Vs” of Big Data which are also termed as the characteristics of Big Data.

- **Volume:** With increasing dependence on technology, data is producing at a large volume. Common examples are data being produced by various social networking sites, sensors, scanners, airlines and other organizations.
- **Velocity:** Huge amount of data is generated per second. It is estimated that by the end of 2020, every individual will produce 3mb data per second. This large volume of data is being generated with a great velocity.
- **Variety:** The data being produced by different means is of three types:
 - **Structured Data:** It is the relational data which is stored in the form of rows and columns.
 - **Unstructured Data:** Texts, pictures, videos etc. are the examples of unstructured data which can't be stored in the form of rows and columns.
 - **Semi Structured Data:** Log files are the examples of this type of data.

- **Veracity:** The term Veracity is coined for the inconsistent or incomplete data which results in the generation of doubtful or uncertain Information. Often data inconsistency arises because of the volume or amount of data e.g. data in bulk could create confusion whereas less amount of data could convey half or incomplete Information.
- **Value:** After having the 4 V's into account there comes one more V which stands for Value!. Bulk of Data having no Value is of no good to the company, unless you turn it into something useful. Data in itself is of no use or importance but it needs to be converted into something valuable to extract Information. Hence, you can state that Value! is the most important V of all the 5V's

Components of Hadoop:

Hadoop has three components:

- **HDFS:** Hadoop Distributed File System is a dedicated file system to store big data with a cluster of commodity hardware or cheaper hardware with streaming access pattern. It enables data to be stored at multiple nodes in the cluster which ensures data security and fault tolerance.
- **Map Reduce :** Data once stored in the HDFS also needs to be processed upon. Now suppose a query is sent to process a data set in the HDFS. Now, Hadoop identifies where this data is stored, this is called Mapping. Now the query is broken into multiple parts and the results of all these multiple parts are combined and the overall result is sent back to the user. This is called reduce process. Thus while HDFS is used to store the data, Map Reduce is used to process the data.
- **YARN :** YARN stands for Yet Another Resource Negotiator. It is a dedicated operating system for Hadoop which manages the resources of the cluster and also functions as a framework for job scheduling in Hadoop. The various types of scheduling are First Come First Serve, Fair Share Scheduler and Capacity Scheduler etc. The First Come First Serve scheduling is set by default in YARN.

Q. How the components of Hadoop make it as a solution for Big Data?

Hadoop Distributed File System: In our local PC, by default the block size in Hard Disk is 4KB. When we install Hadoop, the HDFS by default changes the block size to 64 MB. Since it is used to store huge data. We can also change the block size to 128 MB. Now HDFS works with Data Node and Name Node. While Name Node is a master service and it keeps the metadata as for on which commodity hardware, the data is residing, the Data Node stores the actual data. Now, since the block size is of 64 MB thus the storage required to store metadata is reduced thus making HDFS better. Also, Hadoop stores three copies of every dataset at three different locations. This ensures that the Hadoop is not prone to single point of failure.

Map Reduce: In the simplest manner, it can be understood that MapReduce breaks a query into multiple parts and now each part process the data coherently. This parallel execution helps to execute a query faster and makes Hadoop a suitable and optimal choice to deal with Big Data.

YARN: As we know that Yet Another Resource Negotiator works like an operating system to Hadoop and as operating systems are resource managers so YARN manages the resources of Hadoop so that Hadoop serves big data in a better way.

Hadoop Versions:

Till now there are three versions of Hadoop as follows.

Hadoop 1:

This is the first and most basic version of Hadoop. It includes Hadoop Common, Hadoop Distributed File System (HDFS), and Map Reduce.

Hadoop 2:

The only difference between Hadoop 1 and Hadoop 2 is that Hadoop 2 additionally contains YARN (Yet Another Resource Negotiator). YARN helps in resource management and task scheduling through its two daemons namely job tracking and progress monitoring.

Hadoop 3:

This is the recent version of Hadoop. Along with the merits of the first two versions, Hadoop 3 has one most important merit. It has resolved the issue of single point failure by having

multiple name nodes. Various other advantages like erasure coding, use of GPU hardware and Dockers makes it superior to the earlier versions of Hadoop.

Economically Feasible: It is cheaper to store data and process it than it was in the traditional approach. Since the actual machines used to store data are only commodity hardware.

Easy to Use: The projects or set of tools provided by Apache Hadoop are easy to work upon in order to analyze complex data sets.

Open Source: Since Hadoop is distributed as an open source software under Apache License, so one does not need to pay for it, just download it and use it.

Fault Tolerance: Since Hadoop stores three copies of data, so even if one copy is lost because of any commodity hardware failure, the data is safe. Moreover, as Hadoop version 3 has multiple name nodes, so even the single point of failure of Hadoop has also been removed.

Scalability: Hadoop is highly scalable in nature. If one needs to scale up or scale down the cluster, one only needs to change the number of commodity hardware in the cluster.

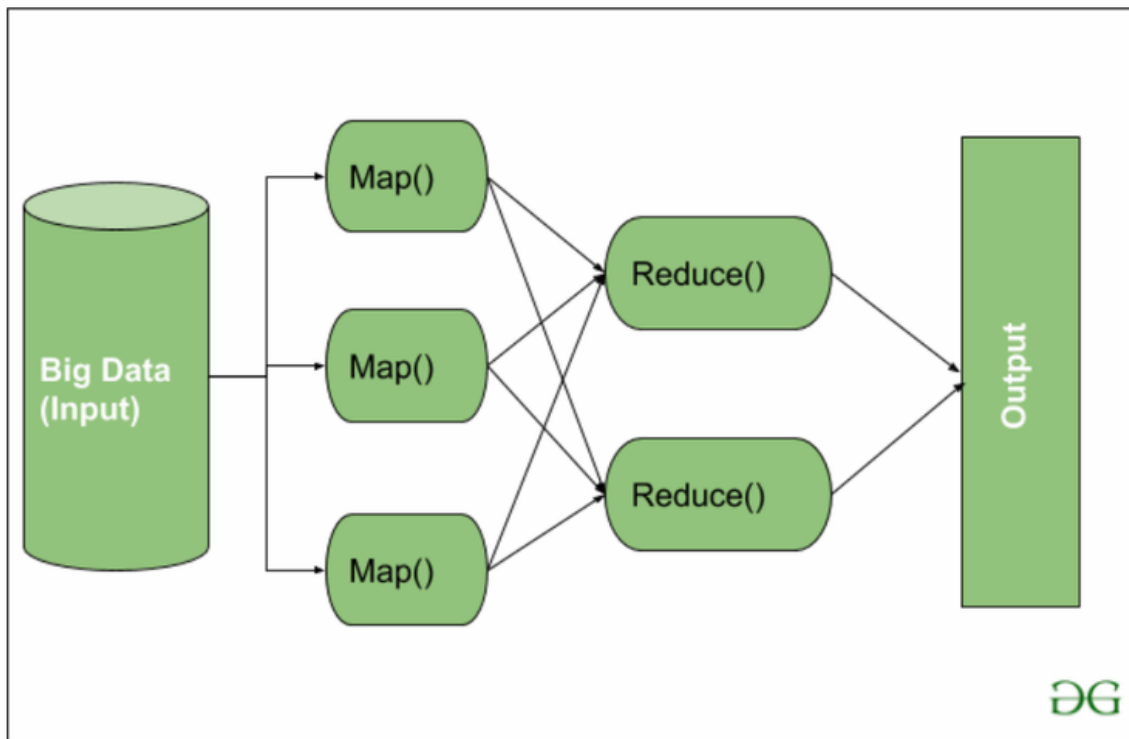
Distributed Processing: HDFS and Map Reduce ensures distributed storage and processing of the data.

Locality of Data: This is one of the most alluring and promising features of Hadoop. In Hadoop, to process a query over a data set, instead of bringing the data to the local computer we send the query to the server and fetch the final result from there. This is called data locality.

Map Reduce in Hadoop

One of the three components of Hadoop is Map Reduce. The first component of Hadoop that is, Hadoop Distributed File System (HDFS) is responsible for storing the file. The second component that is, Map Reduce is responsible for processing the file.

MapReduce has mainly 2 tasks which are divided phase-wise. In first phase, Map is utilised and in next phase Reduce is utilised.



Suppose there is a word file containing some text. Let us name this file as sample.txt. Note that we use Hadoop to deal with huge files but for the sake of easy explanation over here, we are taking a text file as an example. So, let's assume that this sample.txt file contains few lines as text. The content of the file is as follows:

Hello I am Ram

How can I help you

How can I assist you

Are you an engineer

Are you looking for coding

Are you looking for interview questions

what are you doing these days

what are your strengths

Hence, the above 8 lines are the content of the file. Let's assume that while storing this file in Hadoop, HDFS broke this file into four parts and named each part as first.txt, second.txt, third.txt, and fourth.txt. So, you can easily see that the above file will be divided into four equal parts and each part will contain 2 lines. First two lines will be in the file first.txt, next two lines in second.txt, next two in third.txt and the last two lines will be stored in fourth.txt. All these files will be stored in Data Nodes and the Name Node will contain the metadata about them. All this is the task of HDFS. Now, suppose a user wants to process this file. Here is what Map-Reduce comes into the picture. Suppose this user wants to run a query on this sample.txt. So, instead of bringing sample.txt on the local computer, we will send this query on the data. To keep a track of our request, we use Job Tracker (a master service). Job

Tracker traps our request and keeps a track of it. Now suppose that the user wants to run his query on sample.txt and want the output in result.output file

HBase Model in Hadoop

HBase :

HBase is a database that is an open-source platform and it is the implementation of storage architecture by Google's Big Table. The HBase database is column-oriented thus it makes it unique from other databases. One of the unique qualities of Hbase is it doesn't care about data types because we can store different data types of data for the same column in different rows. It contains different sets of tables that maintain the data in key-value format. Hbase is best suitable for sparse data sets which are very common in the case of big data. It can be used to manage structured and semi-structured data and it has many built-in features such as:

Scalability

Versioning

Compression

Garbage collection

There are two types of data storage mediums:

Row-Oriented

Column-Oriented

Row-Oriented :

In the Row-Oriented data storage approach, the data is stored and retrieved one row at a time. This could lead to several problems, suppose we want only some part of the data from the row but according to this approach you have to retrieve the complete row even if you don't need it. Apart from that this approach also serves to get help in the case of the OLTP systems operation and it helps in easy to read and write the records. But it is less efficient in the case when we perform operations on a complete database.

Column-Oriented :

In the column-Oriented data storage approach, the data is stored and retrieved based on the columns. Thus the problem which we were facing in the case of the row-oriented approach has been solved because in the column-oriented approach we can filter out the data which are required to us from the whole set of data with the help of corresponding columns. In the column-oriented approach, the read and write operations are slower than others but it can be

efficient while performing operations on the entire database and hence it permits very high compression rates.

Preference Of HBase :

Hbase can handle large databases very easily as compare to other databases and perform the operations in an efficient manner.

The data expected in Hbase are highly structured and can easily fit in the well-defined schema.

It is easy and suitable for low latency operations.

It provides access to the particular row from thousands of records.

Data in Hbase can be accessed through shell commands or through client API in java.

Other databases get extremely slow while large databases but in the case of Hbase it handles the large databases in a finite manner.

Advantages of HBase :

Hbase provides great functionality for analytics in association with Hadoop MapReduce.

It is capable of handling a very large amount of databases.

Hbase provides the facility of sharing the database with other users.

There are many operations like data reading and processing that will take less amount of time as compared to other traditional databases.

Whenever there is a failure or load sharing problem arises Hbase can automatically recover from the problem because it is internally distributed.

In Hbase scalability is supported in both linear and modular forms.

It provides a large consistency.

Problems in HBase :

In HBase the memory blocks requirements and hardware Requirement for allocation during an operation is expensive.

To store the Large file which is in binary format is very difficult in HBase.

There is no query optimizer present in HBase due to which it is unable to perform SQL functions and it also doesn't support SQL structures.

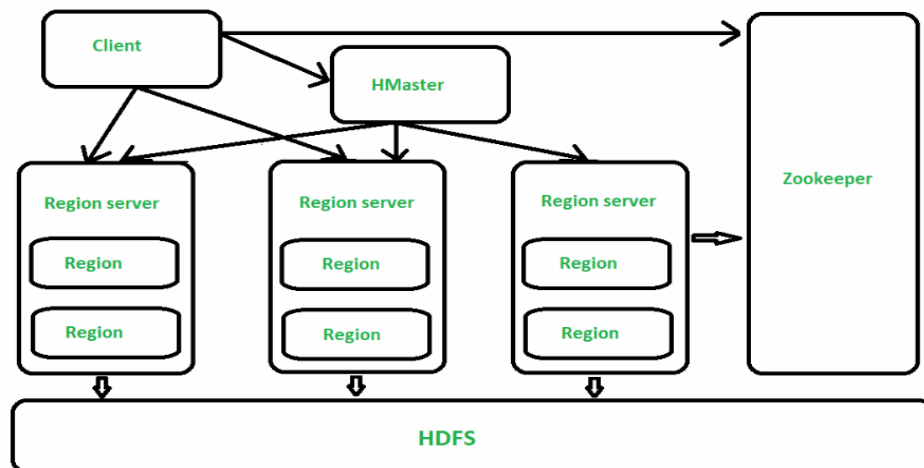
Though there is a lot of functionality provided by Hbase it can't serve as a complete replacement of the traditional models because some of the traditional features are still not supported by HBase.

Hbase does not provide any transaction support.

HBase does not support any default indexing functionality.

Architecture of HBase:

HBase architecture has 3 main components: HMaster, Region Server, Zookeeper.



All the 3 components are described below:

HMaster –

The implementation of Master Server in HBase is HMaster. It is a process in which regions are assigned to region server as well as DDL (create, delete table) operations. It monitor all Region Server instances present in the cluster. In a distributed environment, Master runs several background threads. HMaster has many features like controlling load balancing, failover etc.

Region Server –

HBase Tables are divided horizontally by row key range into Regions. Regions are the basic building elements of HBase cluster that consists of the distribution of tables and are comprised of Column families. Region Server runs on HDFS DataNode which is present in Hadoop cluster. Regions of Region Server are responsible for several things, like handling, managing, executing as well as reads and writes HBase operations on that set of regions. The default size of a region is 256 MB.

Zookeeper –

It is like a coordinator in HBase. It provides services like maintaining configuration information, naming, providing distributed synchronization, server failure notification etc. Clients communicate with region servers via zookeeper.

Advantages of HBase –

Can store large data sets

Database can be shared

Cost-effective from gigabytes to petabytes

High availability through failover and replication

Disadvantages of HBase –

No support SQL structure

No transaction support

Sorted only on key

Memory issues on the cluster

Comparison between HBase and HDFS:

HBase provides low latency access while HDFS provides high latency operations.

HBase supports random read and writes while HDFS supports Write once Read Many times.

HBase is accessed through shell commands, Java API, REST, Avro or Thrift API while HDFS is accessed through MapReduce jobs.

Features of HBase architecture :

- **Distributed and Scalable:** HBase is designed to be distributed and scalable, which means it can handle large datasets and can scale out horizontally by adding more nodes to the cluster.

- **Column-oriented Storage:** HBase stores data in a column-oriented manner, which means data is organized by columns rather than rows. This allows for efficient data retrieval and aggregation.
- **Hadoop Integration:** HBase is built on top of Hadoop, which means it can leverage Hadoop's distributed file system (HDFS) for storage and MapReduce for data processing.
- **Consistency and Replication:** HBase provides strong consistency guarantees for read and write operations, and supports replication of data across multiple nodes for fault tolerance.
- **Built-in Caching:** HBase has a built-in caching mechanism that can cache frequently accessed data in memory, which can improve query performance.
- **Compression:** HBase supports compression of data, which can reduce storage requirements and improve query performance.
- **Flexible Schema:** HBase supports flexible schemas, which means the schema can be updated on the fly without requiring a database schema migration.

Introduction to emerging database technologies-

Cloud Databases:

What is a cloud database?

A cloud database is an organized and managed collection of data in an IT system that resides on a public, private or hybrid cloud computing platform. From an overall design and functionality perspective, a cloud database is no different than an on-premises one that runs on an organization's own data center systems. The biggest difference between them lies in how the database is deployed and managed.

For example, the same database appears identical to end users and applications, whether it's on premises or in the cloud. Depending on the particular database software that's used, cloud databases can store structured, unstructured or semistructured data, just as their on-premises counterparts do.

But using a cloud database changes the responsibilities of IT and data management teams. Cloud vendors install and manage the underlying system infrastructure and, in managed

services environments, the database platform. That reduces the routine management work traditionally done by IT operations workers and database administrators (DBAs). A DBA can then take on other tasks, such as optimizing databases for applications and tracking the usage and cost of cloud database systems.

A Mobile database

A Mobile database is a database that can be connected to a mobile computing device over a mobile network (or wireless network). Here the client and the server have wireless connections. In today's world, mobile computing is growing very rapidly, and it is huge potential in the field of the database. It will be applicable on different-different devices like android based mobile databases, iOS based mobile databases, etc. Common examples of databases are Couch base Lite, Object Box, etc.

Features of Mobile database :

Here, we will discuss the features of the mobile database as follows.

A cache is maintained to hold frequent and transactions so that they are not lost due to connection failure.

As the use of laptops, mobile and PDAs is increasing to reside in the mobile system.

Mobile databases are physically separate from the central database server.

Mobile databases resided on mobile devices.

Mobile databases are capable of communicating with a central database server or other mobile clients from remote sites.

With the help of a mobile database, mobile users must be able to work without a wireless connection due to poor or even non-existent connections (disconnected).

A mobile database is used to analyze and manipulate data on mobile devices.

Mobile Database typically involves three parties :

Fixed Hosts –

It performs the transactions and data management functions with the help of database servers.

Mobiles Units –

These are portable computers that move around a geographical region that includes the cellular network that these units use to communicate to base stations.

Base Stations –

These are two-way radios installation in fixed locations, that pass communication with the mobile units to and from the fixed hosts.

Limitations :

Here, we will discuss the limitation of mobile databases as follows.

- It has Limited wireless bandwidth.
- In the mobile database, Wireless communication speed.
- It required Unlimited battery power to access.
- It is Less secured.
- It is Hard to make theft-proof.

SQLite Database:

SQLite is a highly efficient, serverless, and self-contained SQL database engine that stands out for its simplicity and ease of integration. Designed to be embedded within applications, SQLite eliminates the need for separate database server processes and complex configurations.

What is SQLite?

SQLite is a lightweight, serverless, self-contained, and highly reliable SQL database engine. It is widely used due to its simplicity, ease of setup and zero-configuration nature.

SQLite is an in-process library that implements a self-contained, serverless, zero-configuration, transactional SQL database engine. It is a popular choice as an embedded database for local/client storage in application software such as web browsers. It is also used in many other applications that need a lightweight, embedded database.

SQLite is ACID-compliant and implements most of the SQL standards, using a dynamically and weakly typed SQL syntax that does not guarantee domain integrity.

To use SQLite in a C/C++ program, you can use the SQLite3 API, which provides a lightweight, simple, self-contained, high-reliability, full-featured, and SQL database engine. The API is implemented as a library of C functions that can be called from your program.

One of the main benefits of using SQLite is that it is very easy to get started with. To create a new database in SQLite, you simply need to create a new file on your filesystem and connect to it using the SQLite3 API.

Why Use SQLite?

There are several reasons why you might choose to use SQLite in your project:

Ease of use: SQLite is very easy to get started with, as it requires no setup or configuration. You can simply include the library in your project and start using it.

Embeddability: SQLite is designed to be embedded into other applications. It is a self-contained, serverless database engine, which means you can include it in your application without the need for a separate database server.

Lightweight: SQLite is a very lightweight database engine, with a small library size (typically less than 1MB). This makes it well-suited for use in applications where the database is embedded directly into the application binary, such as mobile apps.

Serverless: As mentioned earlier, SQLite is a serverless database engine, which means there is no need to set up and maintain a separate database server process. This makes it easy to deploy and manage, as there are no additional dependencies to worry about.

Cross-platform: SQLite is available on many platforms, including Linux, macOS, and Windows, making it a good choice for cross-platform development.

Standalone: SQLite stores all of the data in a single file on the filesystem, which makes it easy to copy or backup the database.

High reliability: SQLite has been widely tested and used in production systems for many years, and has a reputation for being a reliable and robust database engine.

Apache Spark

Apache Spark is a lightning-fast cluster computing technology, designed for fast computation. It is based on Hadoop MapReduce and it extends the MapReduce model to efficiently use it for more types of computations, which includes interactive queries and stream processing. The main feature of Spark is its in-memory cluster computing that increases the processing speed of an application.

Spark is designed to cover a wide range of workloads such as batch applications, iterative algorithms, interactive queries and streaming. Apart from supporting all these workload in a respective system, it reduces the management burden of maintaining separate tools.

Features of Apache Spark

Apache Spark has following features.

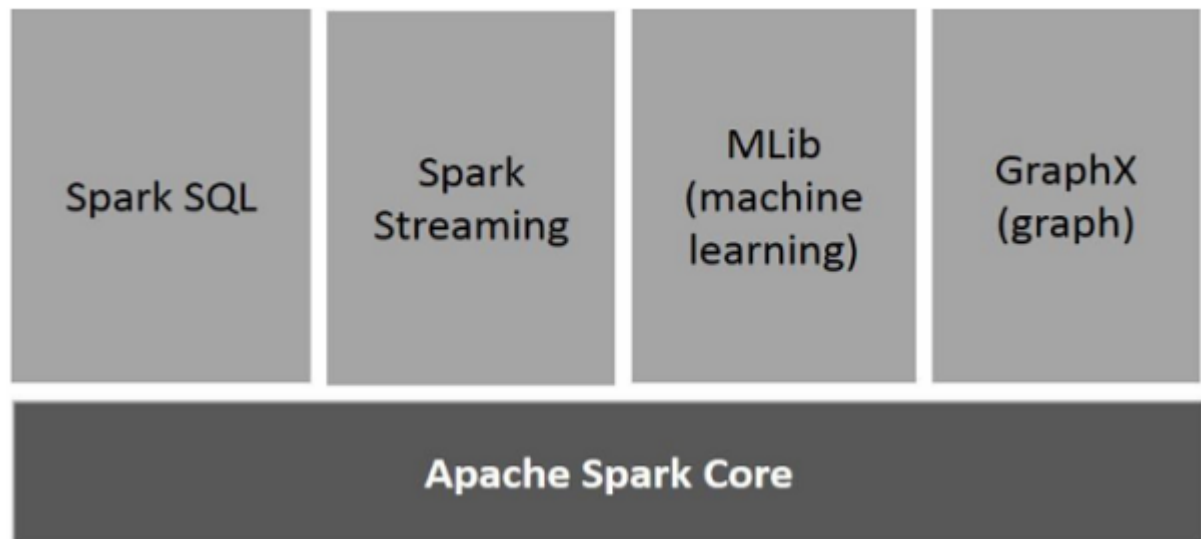
Speed – Spark helps to run an application in Hadoop cluster, up to 100 times faster in memory, and 10 times faster when running on disk. This is possible by reducing number of read/write operations to disk. It stores the intermediate processing data in memory.

Supports multiple languages – Spark provides built-in APIs in Java, Scala, or Python. Therefore, you can write applications in different languages. Spark comes up with 80 high-level operators for interactive querying.

Advanced Analytics – Spark not only supports ‘Map’ and ‘reduce’. It also supports SQL queries, Streaming data, Machine learning (ML), and Graph algorithms.

Components of Spark

The following illustration depicts the different components of Spark.



Apache Spark Core

Spark Core is the underlying general execution engine for spark platform that all other functionality is built upon. It provides In-Memory computing and referencing datasets in external storage systems.

Spark SQL

Spark SQL is a component on top of Spark Core that introduces a new data abstraction called SchemaRDD, which provides support for structured and semi-structured data.

Spark Streaming

Spark Streaming leverages Spark Core's fast scheduling capability to perform streaming analytics. It ingests data in mini-batches and performs RDD (Resilient Distributed Datasets) transformations on those mini-batches of data.

MLlib (Machine Learning Library)

MLlib is a distributed machine learning framework above Spark because of the distributed memory-based Spark architecture. It is, according to benchmarks, done by the MLlib developers against the Alternating Least Squares (ALS) implementations. Spark MLlib is nine times as fast as the Hadoop disk-based version of Apache Mahout (before Mahout gained a Spark interface).

GraphX

GraphX is a distributed graph-processing framework on top of Spark. It provides an API for expressing graph computation that can model the user-defined graphs by using Pregel abstraction API. It also provides an optimized runtime for this abstraction.

Uses :

Speed

Spark executes very fast by caching data in memory across multiple parallel operations. The main feature of Spark is its in-memory engine that increases the processing speed; making it up to 100 times faster than MapReduce when processed in-memory, and 10 times faster on disk, when it comes to large scale data processing. Spark makes this possible by reducing the number of reading/writing to disk operations.

Real-time stream processing

Apache Spark can handle real-time streaming along with the integration of other frameworks. Spark ingests data in mini-batches and performs RDD transformations on those mini-batches of data.

Supports Multiple Workloads

Apache Spark can run multiple workloads, including interactive queries, real-time analytics, machine learning, and graph processing. One application can combine multiple workloads seamlessly.

Increased Usability

The ability to support several programming languages makes it dynamic. It allows you to quickly write applications in Java, Scala, Python, and R; giving you a variety of languages for building your applications.

Advanced Analytics

Spark supports SQL queries, machine learning, stream processing, and graph processing.